DOCS AS CODE

WHAT INFORMATION DESIGNERS NEED TO KNOW ABOUT THE NEW WAVE OF API DOCUMENTATION

While the quality of the user manual isn't likely to factor into a consumer's choice of one toaster over another, the same can no longer be said for APIs.

by Sybil Scott

INTRODUCTION

Creating and maintaining API documentation can be costly, and it is sometimes hard to judge the return on investment. Therefore, not much attention has been paid to quality documentation historically; having something--anything--was good enough.

That's changing, however. As Uddin and Robillard stated in their *IEEE Software* article, "How API Documentation Fails" (2015), "Because good documentation can help developers work efficiently, it can even serve to promote the API. In contrast, documentation that doesn't meet its readers' expectations can lead to frustration, major loss of time, and even abandonment of the API."

Companies are realizing that there are marketing (i.e. profit) considerations around documentation; as a result, efforts to make it both easy to create and easy to use are ramping up.

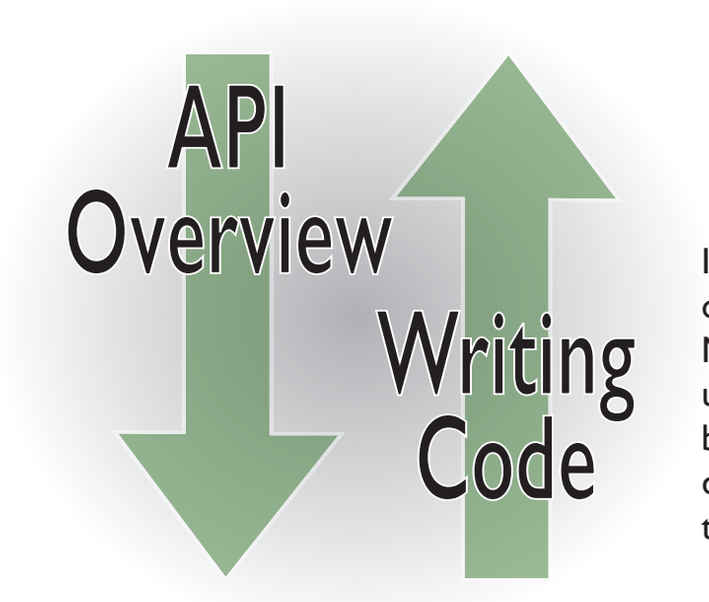
Historically, developers have a reputation of being bad at documentation, disinterested in it, or both (Thomchick, 2018).

Enter docs as code.

Since developers are both creators and end users of API documentation, this new philosophy seeks to make documentation as much like code as possible. It allows developers to create and use documentation using the methods and tools with which they are already comfortable and efficient. The task should be less painful because it's closer to their usual processes and skill set.

MAKING API DOCUMENTATION EASY TO USE

Further Audience Consideration



While developers are the main audience of API docs, there is still a wide range of factors to consider.

Good documentation needs to be usable by both novice and experienced developers, and by those both familiar and unfamiliar with the specific API being used.

That means providing sufficient introductory and overview information without that content getting in the way of advanced users who need to pinpoint specific information.

In addition, as outlined in the observation study carried out by Meng et al. (2019) on how developers use documentation, the docs need to be usable by developers who engage different strategies as they approach the API:

- **Systematic developers** tend to read more overview information up front to understand the API more thoroughly before starting a task.
- **Opportunistic developers** try to start coding immediately in a trial-and-error fashion, then seek information (specifically, code examples) that directly relates to the task.
- **Pragmatic developers** use a combination of systematic and opportunistic approaches.

MAKING API DOCUMENTATION EASY TO USE

Documentation Issues

In Uddin and Robillard's research on the documentation issues developers face (2015), a few presentation factors surfaced, all having to do with the needed content being too hard to find.

- **Bloat** and **Excessive Structural Information:** too much information means users can't separate the content they need from that they don't. Restating structural information found elsewhere buries the important content.
- **Fragmentation:** when the information isn't arranged logically and is therefore spread out over multiple pages. Leads to frustrated searching and too many clicks.
- **Tangled information:** when multiple usage scenarios are included in one description so it's hard to separate out the needed scenario from the others.

MAKING API DOCUMENTATION EASY TO USE

Information Design Considerations

Navigation

Good navigation ensures that novice users have all of the information they need while also ensuring experts can skip straight to the helpful content.

- **Web delivery** is preferable to a PDF format (Eker and Rondeau, 2016) for many reasons, including better navigation options
- **Well categorized content** means users will find the content they're look for in the most logical section, meaning less jumping from page to page
- **A robust search tool** is important

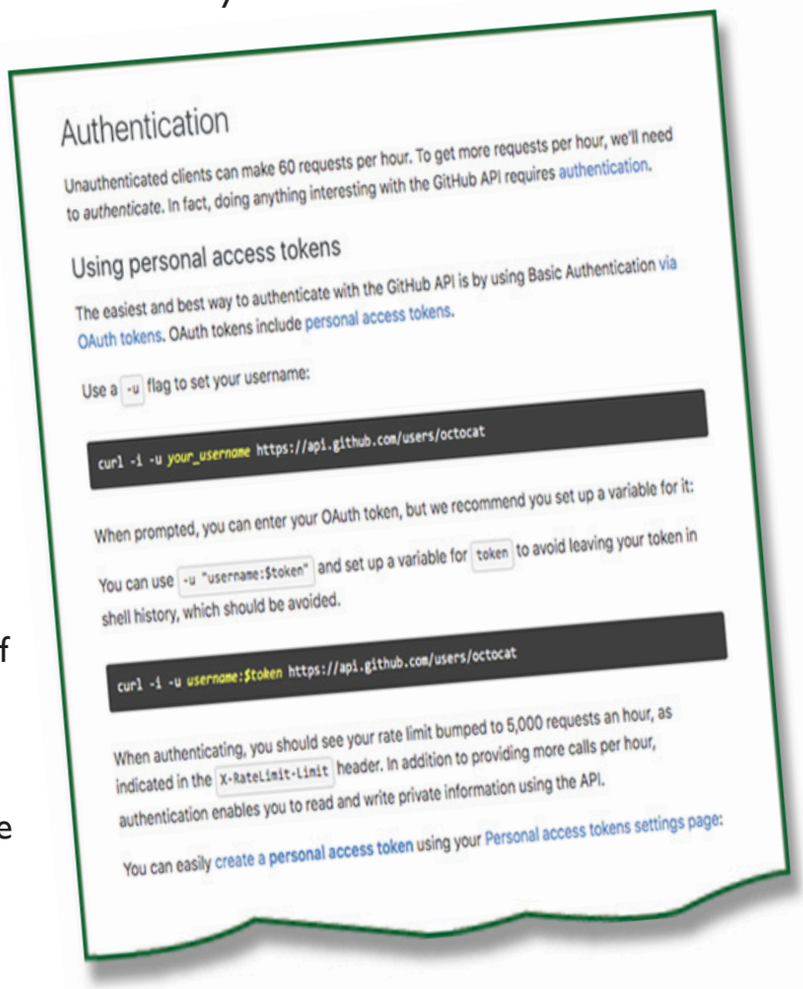
MAKING API DOCUMENTATION EASY TO USE

Information Design Considerations

Layout

The Layout of each section must **easily distinguish different types of content,** such as descriptions, data models, and commands. **Code examples** are one of the most useful aspects of documentation and should easily stand out from other content. They should also have a format that is easily copied and pasted (Meng et al., 2019).

In addition, the basic concepts of good design are in play on every page, but especially on introduction pages, which will likely be more text-heavy. **Clear headers, effective white space,** and adherence to the design principles of **contrast, repetition, alignment, and proximity** (Williams, 2015) will make for the most useful and usable content.



Authentication

Unauthenticated clients can make 60 requests per hour. To get more requests per hour, we'll need to authenticate. In fact, doing anything interesting with the GitHub API requires authentication.

Using personal access tokens

The easiest and best way to authenticate with the GitHub API is by using Basic Authentication via OAuth tokens. OAuth tokens include personal access tokens.

Use a `-u` flag to set your username:

```
curl -i -u your_username https://api.github.com/users/octocat
```

When prompted, you can enter your OAuth token, but we recommend you set up a variable for it:

You can use `-u "username:$token"` and set up a variable for `token` to avoid leaving your token in shell history, which should be avoided.

```
curl -i -u username:$token https://api.github.com/users/octocat
```

When authenticating, you should see your rate limit bumped to 5,000 requests an hour, as indicated in the `X-RateLimit-Limit` header. In addition to providing more calls per hour, authentication enables you to read and write private information using the API.

You can easily create a personal access token using your Personal access tokens settings page:

MAKING API DOCUMENTATION EASY TO CREATE/MAINTAIN

While less directly related to information designers, it's good to understand how docs as code manifests in ways that motivate and help developers (the subject matter experts of APIs) write documentation by using the same systems, processes, and workflows as coding.

```
| Name | Definition |
| ---- | ----- |
| Positive| Greater than zero |
| Negative| Less than zero |
```

→

Name	Definition
Positive	Greater than zero
Negative	Less than zero

- Use a lightweight markup language such as Markdown, a simplified form of HTML (*More about Markdown*, n.d.).
    - Uses a code-like syntax
    - Works in text-file formats using any preferred code editor
    - Treats the files with the same workflow and routing as code
    - Focuses on content instead of formatting
- Store docs in the same version control repository as the code itself, which is more convenient and encourages documenting while coding instead of after the fact. (Johnson, 2017)
- Automate when possible: emerging technologies like Swagger can automate certain processes, allowing developers to focus on the more complex content. (*Swagger,* n.d.)
- Document iteratively and incrementally by constantly reviewing, testing, updating, and publishing docs as opposed to leaving the documentation for the end of the process and considering it one-and-done. (*Principle 9*, n.d.)

REFERENCES

Docs as Code—Write the Docs. (n.d.). Retrieved February 22, 2020, from https://www.writethedocs.org/guide/docs-as-code/

Johnson, Tom. (2017, June 2). Limits to the idea of treating docs as code. I'd Rather Be Writing. https://idratherbewriting.com/2017/06/02/when-docs-are-not-like-code/

Margaret Eker, Jennifer Rondeau - Docs as Code: The Missing Manual. (n.d.). Retrieved February 24, 2020, from https://www.youtube.com/watch?v=JvRd7MmAxPw

Meng, M., Steinhardt, S., & Schubert, A. (2019). How Developers Use API Documentation: An Observation Study. 11.

More about Markdown. (n.d.). Documenting APIs. Retrieved March 3, 2020, from https://idratherbewriting.com/learnapidoc/pubapis_markdown.html

Principle 9: Iterate and increment on content following an agile approach. (n.d.). I'd Rather Be Writing. Retrieved March 3, 2020, from https://idratherbewriting.com/simplifying-complexity/iterative-and-increment-on-content.html

The Best APIs are Built with Swagger Tools | Swagger. (n.d.). Retrieved March 7, 2020, from https://swagger.io/

Thomchick, R. (2018). Improving access to API documentation for developers with Docs-as-Code-as-a-service. Proceedings of the Association for Information Science and Technology, 55(1), 908–910. https://doi.org/10.1002/pra2.2018.14505501171

Uddin, G., & Robillard, M. P. (2015). How API Documentation Fails. IEEE Software, 32(4), 68–75. https://doi.org/10.1109/MS.2014.80

Williams, Robin. *The non-designer's design book: design and typographic principles for the visual novice*. Pearson Education, 2015

XML attributes and examples: Learn API Documentation with JSON and XML. (n.d.). Retrieved February 23, 2020, from https://www.linkedin.com/learning/learn-api-documentation-with-json-and-xml/xml-attributes-and-examples